

Task 3 - Modelling

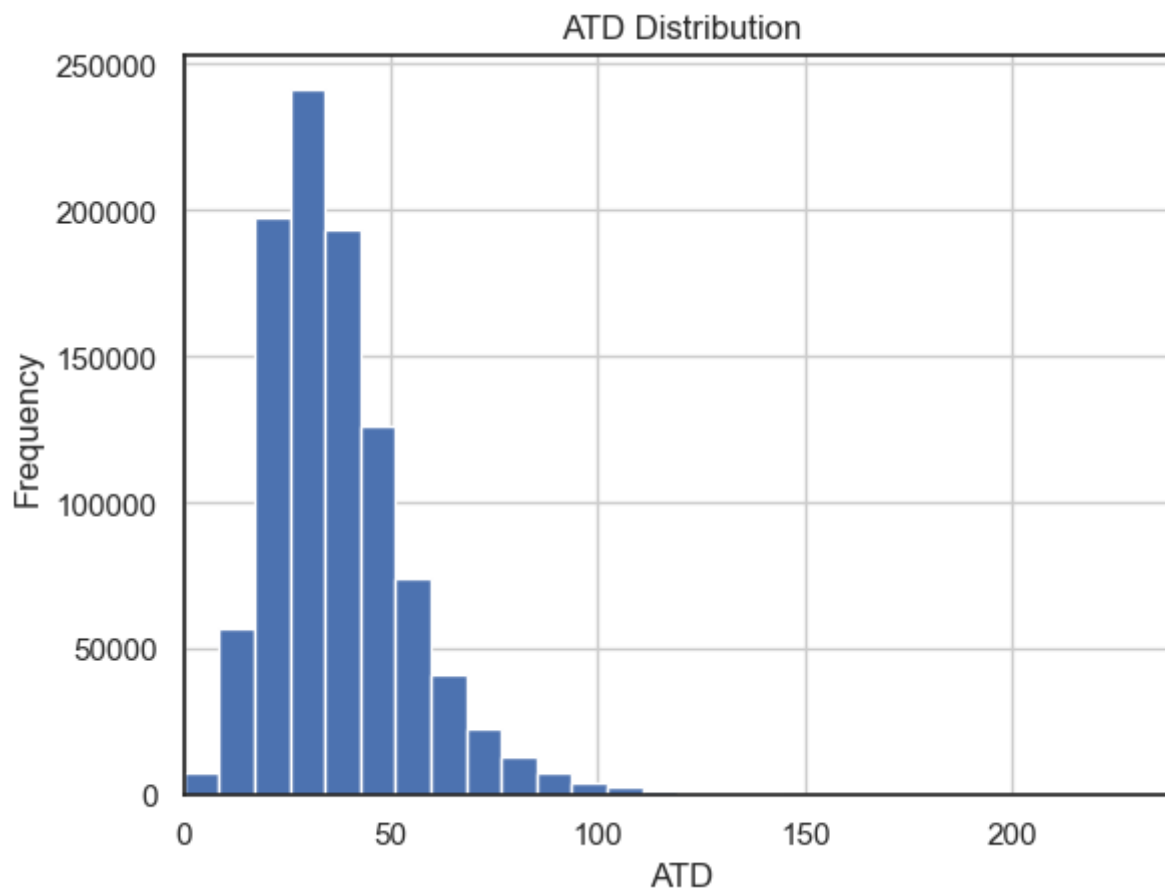
Since it was a bonus, I focused most of my efforts on the other tasks, but managed to produce a model that works as better-than-average benchmark for any potential future modeling.

For a deeper dive into my reasoning process for deciding on KPIs, data transformations and modeling decisions, please refer to my notebooks folder in the GIT repo.

1. Target Definition

ATD was precomputed and included as such in the provided dataset

Studying the distribution of ATD, it was clear it was highly dominated by high valued ATD outliers, making it significantly biased and right skewed. With values going all the way up to 8515 (the images is limited to the (0,240) range), it was clear a transformation was needed.



After thinking about it, I proceeded with a logarithmic transformation (specifically $\log(1+x)$) for 2 main reasons:

1. Its very easy to revert to the original scale
2. It centers and unbias the distribution which in this particular case makes it so the gradient isn't dominated by huge, very probably outlier, values.

2. Data Cleaning and Preprocessing

The dataset was provided almost perfectly clean. The only changes I felt the need to make were:

1. Set all time columns to local timezone for interpretability
2. Depending on the category, treat NAN values as orders where the user picked up their meal directly from the store: so *courierflow NANs became _Unassigned*, *driver_uuid* too, and both distances were imputed with the mean of the data grouped by territory, hour, day of week and geo archetype to ensure consistency.

3. Feature Engineering

In order to improve the model, I applied a couple of basic transformations to some key variables (although in the case of ensemble models they are not that necessary):

1. Log transformation of skewed, outliers packed, variables like distances. Distance, intuitively, directly affects ATD.
2. One-hot encoding for categorical variables (turning NAN and missing into important features based on the assumption that they were very probably orders that the user picked in the store). My assumption is that all categorical columns I received should affect ATD one way or the other: from merchant surface being a proxy for unobserved qualities of the restaurant, to geo_archetype telling us something about the strategy of UberEats, I left them all.
3. Sine, cosine transformations of hours to make sure steps from 22-23, 23-00 and 0-1 are linear instead of having a huge step from 23-00.

I also went into some more creative feature generation with:

1. Generated bins for drivers based on their observed experience levels (number of trips) -> More experienced drivers probably have a direct influence on ATD.
2. Extracted hour and day of week from datetime
3. Used hour and day_of_week to define peak atd hours (probably traffic/demand related), peak volume hours (probably supply related) and is_weekend. -> Time and day of week intuitively affect the ATD, in the early morning there won't be many orders but also, there won't be many drivers to take care of the few that come up, in the afternoons, it feels like there's never enough drivers to take care of the big demand in a traffic-ridden city.
4. Used the log of distances to define a ratio between both and Winsorized (essentially clipped the extremes to less extreme values) them to reduce extreme values.

This practices, of log transforming and winsorizing, together with one hot encoding, help the model learn better splits. Without working out the outliers, the splits can much more probably come from extreme values without there necessarily being value in that split.

4. Model Selection and Training

Given the size of the dataset and the non-linear (at least not apparent to me), I decided to train an ensemble model to predict `ATD_log`.

Ensemble models are at the frontier between interpretability and accuracy, being models that while much more accurate than other simpler, less flexible models, they're also much more interpretable than Neural Networks, cheaper and easier to train and require less data.

Given the times constraints and the bonus nature of the task, and given that I only needed the model to integrate it into the dashboard, I didn't test other models. My approach was looking for the best set of parameters to use in xGboost, using GridSearch to automatically search over a grid of hyperparameters.

To compare the models, I optimized for the MAE which directly measures the average absolute deviation in minutes and is very simple to interpret for non-technical stakeholders. In this context, specially given it is a regressor, I also looked into the distribution of its residuals and the actual vs predicted plot.

In a very simple grid search, I obtained a MAE of 0.28 which roughly translates to 1.32 multiplicative error when transforming back to the original ATD, which although not perfect, is a very decent range for an initial baseline model.

5. Data Splitting and Validation Strategy

The data was split randomly on an 80-20 basis. However, I'm aware of the potential real-world problem this induces. In a more extensive modeling task, aimed at real-world deployment I would have opted for a split where Training uses the 80% older weeks and validation the 20% most recent leaks so that way there are no future labels in training and we respect some seasonality (although the granularity and semi-independent nature of transactions is not the best for that).

In case of cross-validation, the approach would be to use a sliding window so the model is tested on multiple future data.

6. Evaluation metrics

The primary metric is Mean Absolute Error (MAE) since, I already exposed above, it is very easy to interpret. However here are other alternatives I came up with in the generation of metrics:

- Custom **Asymmetrical** (this is key) loss: in this context, penalization of underestimation should be heavier than that of overestimation. This is important from both the operations

and the client side, since in both cases, underestimation causes UberEats to suffer more if it underestimates the real ATD. A simple proposal would be:

$$\text{Loss}(y, \hat{y}) = \begin{cases} \alpha \cdot |y - \hat{y}| & \text{if } \hat{y} < y \text{ (underestimation)} \\ (1 - \alpha) \cdot |y - \hat{y}| & \text{if } \hat{y} \geq y \text{ (overestimation)} \end{cases} \text{ where}$$

- y = ATD (log-transformed)
- $\hat{y}$ = predicted ATD
- $\alpha \in (0.5, 1]$ is the weight of penalization where as the weight linearly increases the ratio of importance between both geometrically increases

Here the gradient is very easy to estimate and the only issue arises with the Hessian (since it becomes zero), yet a very small constant (say $1e-8$) should probably work.

- Assuming we had an SLA and a Cost per minute of delay, training a model to simultaneously learn to predict ATD while also learning to predict breach probability or cost, would be very important for operations since it would also report certain metrics relevant for problem domain.

Now, on traditional evaluation metrics, RMSE penalizes outliers heavily and R^2 is already hard to properly explain to non-technical stakeholders when it is a one-variable model, so when using multiple variable it becomes harder to interpret. However, for the technical team both are relevant.

It is very important to monitor both concept and data drifts, probably by empirically monitoring the distribution of recent batches with previously observed data to raise any alarm. In a blooming market and an unstable country like we have in Latinamerica, data drift and, more importantly, concept drift are very relevant given their plausibility.

7. Integration into Pipeline and Dashboard

1. ETL:

1. Weekly job extracts data with the query created for task 1 and updates the delivery data for the previous week
2. The data is staged in order to be cleaned, ensure its quality and consistency and provide a clear trail for any necessary auditing
3. A feature extraction pipeline, pre-fit, generates the features and finalizes the final training table

2. Model Inferences:

1. The pre-trained model runs only on the new data and outputs $\text{ATD}_{\log}$
2. The predictions are reverted to the original scale by using $\exp \hat{y} - 1$

3. Storage:

1. Predictions are stored together with the features or even as a transformed column of the original data

4. Streamlit Dashboard:

1. Pulls predictions from table and integrates them dynamically into the dashboard.